08 – Roberta Williams

Contacto

845977	Jaime Alconchel Gallego
844699	Mario Casado Fontana
842102	Cristina Embid Martínez
844472	Hugo Naudín López
844459	Miguel Ángel Luquín Guerrero
792310	Julia Quero Pérez
843740	Nicolás Eugenio de Rivas Morillo
844487	Miguel Ángel Royo Miguel

Plan de gestión, análisis, diseño y memoria del proyecto MAGNATE

15 de marzo de 2026

Índice

1. Introducción **2**

2. Organización del proyecto **3**

3. Plan de gestión del proyecto **3**
3.1. Inicio del proyecto . . . 3
3.1.1. Mitigación de riesgos . . . 4
3.2. Control de configuraciones, construcción del software y aseguramiento de la calidad del producto . . . 5
3.3. Calendario del proyecto, división del trabajo, coordinación, comunicaciones, monitorización y seguimiento . . . 7

4. Análisis y diseño del sistema **10**
4.1. Análisis de requisitos . . . 10
4.2. Diseño del sistema . . . 10
4.2.1. Decisiones de diseño . . . 14

5. Memoria del proyecto **15**
5.1. Inicio del proyecto . . . 15
5.2. Ejecución y control del proyecto . . . 15
5.3. Cierre del proyecto . . . 15
5.3.1. Dedicación por integrante . . . 16

A. Diagramas **17**
A.1. Diagrama de componentes y conectores. . . . 17
A.2. Diagrama de estados de la fase de Subasta de un turno del juego Magnate . . . 18
A.3. Diagramas de módulos . . . 18

B. Análisis de riesgos **20**

C. Reglas **22**
C.1. Introducción . . . 22
C.2. Objetivo del juego . . . 22
C.3. Equipamiento . . . 22
C.3.1. Los dados: definición y funcionamiento . . . 22
C.3.2. Tablero . . . 23
C.4. Casillas . . . 23
C.4.1. Propiedades . . . 23
C.4.2. Grupos de propiedades . . . 23
C.4.3. Puentes . . . 23
C.4.4. Servidores . . . 23
C.4.5. Casillas fantasía . . . 24
C.4.6. Vas a la cárcel . . . 24
C.4.7. Cárcel . . . 24
C.4.8. Salida . . . 24
C.4.9. Tranvías . . . 24
C.4.10. Párking . . . 24
C.5. Preparación . . . 24
C.6. Turno de juego . . . 25
C.7. Subastas . . . 25

C.8. Alquileres 25
C.9. Construcciones 25
C.10. Intercambios 25
C.11. Bonificaciones finales 25
C.12. Bancarrota 26

1. Introducción

El presente documento recoge el plan de gestión, análisis, diseño y memoria del proyecto Magnate. Magnate es un sistema que consiste en un juego multiplataforma, para escritorio y web, online y multijugador, con un diseño sencillo, estético y lúdico. Magnate es un juego de mesa por turnos basado en el popular juego Monopoly, tomando inspiración adicional de la saga de videojuegos Mario Party. Magnate busca ser un juego dinámico con alto grado de aleatoriedad, pero sin perder la estrategia del Monopoly original, al que añade más reglas que esperan hacer de la experiencia de juego novedosa y emocionante. Estas incluyen un tablero en dos niveles, un nuevo sistema de cartas, más formas de desplazamiento a lo largo del tablero y un nuevo sistema de puntuación.

El sistema permitirá dos modos de juego: partidas online y partidas locales. Las partidas online se realizarán en salas públicas con 4 usuarios interesados en jugar partidas online. Las partidas locales permitirán el juego de varios jugadores en local y bots, sumando hasta 4 jugadores.

Otros objetivos del sistema son:

- Pausar y reanudar partidas entre dispositivos.
- Asociar cuentas de usuario con la información de las partidas.
- Ofrecer una experiencia atractiva y personalizable, incluyendo una tienda de cosméticos.
- Garantizar la seguridad y privacidad de los usuarios.

El sistema será compatible con el navegador web Chromium 144.0.7559.96 y para aplicaciones de escritorio con el sistema operativo Arch Linux 6.18.6. Se desplegará en varios contenedores Docker en un servidor de la empresa 08 – Roberta Williams. Las tecnologías a utilizar abarcan Godot 4.6, React, Django y PostgreSQL.

El desarrollo del sistema se realizará a lo largo de los meses de febrero, marzo, abril y mayo. Los hitos principales son:

- Entrega intermedia: 6 de abril de 2026. Prototipos sin integración.
- Presentación del sistema: 4 o 5 de mayo de 2026.
- Muestra del sistema al cliente: semana del 11 de mayo de 2026.
- Entrega final y cierre de proyecto: 27 de mayo de 2026.

El precio del sistema y el proyecto suman 30795,50 €. Por dicho precio se entregará lo siguiente a la finalización del proyecto:

- Todo el código desarrollado.
- Documentación de despliegue y ficheros de configuración y Docker Compose para el despliegue.
- Un AppImagen con la versión escritorio, ya compilado.

El cuerpo del documento se estructura como sigue: en la Sección 2 se especifican los integrantes, roles y tareas de la empresa; en la Sección 3 se describe la planificación inicial del proyecto, en cuanto a los recursos necesarios, temporalidad, procedimientos y responsables de las tareas; en la Sección 4 se describe el sistema, enumerando los requisitos, especificando el diseño y tecnologías y justificando las decisiones de diseño tomadas; y en la Sección 5 se recoge la memoria del proyecto. Los apéndices incluyen el Apéndice A, donde se incluyen algunos diagramas; el Apéndice B, con la información complementaria del análisis de riesgos realizado; y el Apéndice C, donde se detallan las reglas del juego Magnate.

2. Organización del proyecto

Los encargados del proyecto son la totalidad de la empresa 08 – Roberta Williams, que conformamos los estudiantes de 5º curso del Programa Conjunto en Matemáticas e Ingeniería Informática de la Universidad de Zaragoza. La creaación del equipo de trabajo surge de manera natural tras las experiencias compartidas a lo largo del grado, que refuerzan nuestra confianza y seguridad en nosotros mismos como equipo y como profesionales.

Basándonos en nuestras habilidades y experiencia previa, se ha decidido conjuntamente el siguiente reparto de cargos y tareas principales, recogido en la Tabla 1.

<table>
<tr><td>Hugo</td><td rowspan="3">backend</td><td></td><td>IA</td><td></td><td></td></tr>
<tr><td>Jaime</td><td rowspan="2">despliegue</td><td>encargado despliegue</td><td>diagramas</td><td>coordinador backend</td></tr>
<tr><td>Mario</td><td>lógica del juego</td><td rowspan="2">diseño de marca</td><td></td></tr>
<tr><td>Miguel Royo</td><td rowspan="5">frontend</td><td rowspan="3">web</td><td>coordinador frontend</td><td rowspan="2">diseño interfaz</td></tr>
<tr><td>Cristina</td><td>encargada calidad web</td><td></td></tr>
<tr><td>Julia</td><td>calidad</td><td>documentación</td><td>gestión del proyecto</td></tr>
<tr><td>Nicolás</td><td rowspan="2">escritorio</td><td>encargado escritorio</td><td>diagramas</td><td rowspan="2">diseño interfaz</td></tr>
<tr><td>Miguel Luquín</td><td>encargado calidad escritorio</td><td></td></tr>
</table>

Tabla 1: Relación de cargos y tareas principales en la empresa 08 – Roberta Williams.

De los roles recogidos en la Tabla 1, queremos destacar las siguientes responsabilidades:

- Directora de proyecto: Julia Quero Pérez
- Coordinador de fronend: Miguel Ángel Royo Miguel
- Coordinador de backend: Jaime Alconchel Gallego
- Coordinadora de calidad: Cristina Embid Martínez
- Coordinador de despliegue: Jaime Alconchel Gallego

3. Plan de gestión del proyecto

En esta sección se describe cómo se llevarán a cabo los distintos procesos o tareas a realizar a lo largo del tiempo, su temporalidad, los recursos a emplear y quién son sus responsables.

3.1. Inicio del proyecto

Se ha priorizado el uso de herramientas abiertas o de software libre para la realización del proyecto, por lo que los recursos materiales y software se reducen a los ya accesibles a los miembros del equipo. Estos incluyen los ordenadores portátiles de cada miembro del equipo y el servidor que posee Mario.

La formación inicial necesaria para los miembros del equipo, por tecnología utilizada, se detalla a continuación:

- Godot 4.6 (implementación del sistema para aplicación de escritorio): Miguel, Miguel y Nicolás buscarán por su cuenta tutoriales de la aplicación.
- React (implementación del sistema para navegador web): Miguel Royo buscará información al respecto, y Cristina le ayudará a buscar tutoriales y proyectos de la librería Phaser para el juego. Julia se encargará de incluir el sonido.
- Django (backend): Los miembros del equipo ya tenían experiencia con la tecnología.
- PostgreSQL (gestor de BDD): Los miembros el equipo ya tenían experiencia con la tecnología.

3.1.1. Mitigación de riesgos

Se han detectado los siguientes riesgos del proyecto:

Ri1 Hay integrantes del equipo que tienen muchos proyectos (y/o asignaturas a la vez).

Ri2 La falta de experiencia trabajando juntos puede afectar al éxito del proyecto.

Ri3 La toma de decisiones se puede bloquear por no estar dispuestos los miembros del equipo a asumir las responsabilidades.

Ri4 Nunca se ha trabajado en un grupo tan grande.

Ri5 El desconocimiento del cliente.

Ri6 El proyecto cuenta con unas funcionalidades que no están muy claramente definidas.

Ri7 Nunca se ha construido un sistema tan complejo como el propuesto.

Ri8 Se ha elegido una tecnología que se conoce poco o nada.

Ri9 Se ha trabajado, o no, con el entorno de despliegue elegido.

Ri10 La fecha de entrega está muy lejos en el tiempo.

Ri11 El proyecto hay que desarrollarlo en dos fases.

Para ellos se proponen las siguientes estrategias de mitigación:

Ri1 Promover la organización individual, evitar el solapamiento en la definición de las tareas y fijar fechas internas para todas las pequeñas tareas. Responsables: Julia, Cristina.

Ri2 Favorecer la comunicación inmediata en los grupos de WhatsApp y Discord. Responsable: Julia.

Ri3 Repartir las tareas conforme surgen. Responsables: Julia, Jaime y Miguel Royo.

Ri4 Fomentar el uso de herramientas y metodologías para la comunicación fluida (WhatsApp, Discord), realizar un reparto de trabajo a nivel de los grupos de trabajo y posteriormente ponerlo en común y conjunto en las sesiones de prácticas con el equipo al completo. Responsables: Julia, Jaime y Miguel Royo.

Ri5 Aprovechar las sesiones de prácticas para consultar el estado del proyecto y solicitar retroalimentación. Responsable: Julia.

Ri6 Establecer requisitos claros. Responsable: Julia.

Ri7 Dividir el proyecto en tareas pequeñas y autocontenidas por equipos y establecer fechas para juntar los resultados. Responsable: Julia.

Ri8 Fomentar la consulta y realización de tutoriales al comienzo del proyecto, previo al desarrollo. Responsable: Miguel Luquín.

Ri9 Revisar proyectos ya realizados con esa misma tecnología y hacer alguna prueba menor antes de configurar el sistema completo. Responsable: Jaime.

Ri10 Poner fechas internas con un margen suficiente para entregar a tiempo el proyecto en las fechas de entrega con el cliente. Responsable: Julia.

Ri11 Establecer fechas intermedias (hitos en Github issues). Responsables: Cristina, Nicolás, Mario.

3.2. Control de configuraciones, construcción del software y aseguramiento de la calidad del producto

Las **herramientas** a utilizar en el proyecto se han elegido por ser aquellas que aportan más profesionalidad y por contar cada miembro con experiencia en su uso.

El código del proyecto se ha desarrollado en el entorno de desarrollo (VisualStudio Code o vim) de la preferencia de cada integrante, y se ha integrado en la plataforma GitHub, utilizando la herramienta git de control de versiones, así como la documentación escrita con LaTeX.

Se ha utilizado la plataforma de gestión de archivos Google Drive y sus filiales Google Sheets y Google Docs para el seguimiento del desempeño, la realización de presupuestos o análisis y compartir de forma preliminar cierta información bibliográfica o de gestión.

Para la creación de diagramas arquitecturales se ha seguido la notación UMLS, y se ha utilizado PlantUML. Para los diagramas de tareas, por otro lado, se ha utilizado una hoja de cálculo de Google Sheets.

Para el diseño de la GUI, en primer lugar se concretó una guía de estilo general de los elementos de la interfaz, y posteriormente se realizan a papel los bocetos de cada pantalla, que tienen su primer prototipo directamente en la tecnología escritorio o web utilizada en el proyecto. Los diseños se hacen siguiendo el sentido estético del equipo de frontend, más desarrollados por algunos de sus miembros que otros, que se centran en las cuestiones de usabilidad. Como referencia para la interfaz se siguen Las 8 reglas de oro de Schneiderman, y se intentan minimizar la interacción del usuario sin comprometer su comprensión. Para la guía de estilo se eligieron un número reducido de colores principales y secundarios y se consensuaron los formatos de los elementos básicos (botones, fondos...) para la consistencia.

Por último, las herramientas de despliegue engloban, a nivel software, contenedores Docker que desplieguen un sistema operativo Alpine Linux; y a nivel hardware, para el despliegue de los contenedores, una máquina propiedad de Mario Casado con las siguientes especificaciones: UGREEN DXP2800 con procesador Intel n100, 8TB de HDD, 1TB de SSD m2 y 16GB de RAM DDR5. Como todo el despliegue se realizará en la misma máquina, los puertos y rutas necesarios para el despliegue aparecerán como variables de entorno en los ficheros yaml, y no será necesario especificar nada más.

Los **responsables** de supervisar la corrección de las configuraciones, construcción del software y calidad del producto son para el código del backend Jaime (el coordinador del backend) y para el código del frontend Miguel (el coordinador del frontend).

La convención de nombrar ficheros y documentos y **estándares de código** se sigue según los estándares de los lenguajes empleados, y según las especificaciones de las herramientas de generación automática de documentación empleadas. A nivel general y común a todos ellos, puede especificarse que el código está escrito en inglés (nombres de variables y comentarios), utilizando el nombrado con camelCase para las clases y el resto de variables con snake_case. Para la generación

de la documentación se contemplaron distintas herramientas para cada módulo (backend, web, escritorio). Para los módulos del backend: el código interno y los websockets utilizan mkdocs con docstrings, y el código de API Rest utiliza swagger. Para los módulos del cliente web se utiliza TypeDoc. Para los módulos del cliente escritorio se utiliza Godoct (ver [3]).

Para el desarrollo de la aplicación para entornos de escritorio se seguirán las buenas prácticas explicadas en [1]. Entre estas, se destacan algunos detalles específicos a las herramientas:

- Signal up call down: Aplicar el patrón de diseño Publicación-Subscripción mediante señales de Godot, manteniendo la jerarquía de nodos.
- Estilo: El lenguaje de programación de la herramienta, GDScript, es muy similar a Python, se recomienda el uso del estándar PEP8 (ver [4]).
- Evitar clases pesadas: Se heredará de clases más ligeras que la clase Node, como RefCounted.

El código se ha organizado en 4 **repositorios** de GitHub: uno para el backend, otro para el frontend del cliente web, otro para el frontend del cliente escritorio y otro para la gestión y documentación. Todos los miembros del equipo tienen acceso a todos los repositorios, y respetan el workflow consensuado en el equipo. Las issues se crean tras una reunión entre los miembros del equipo correspondiente en la que se identifican, definen y reparten las tareas a realizar. Se designa un encargado que cree en Github las issues correspondientes. Para realizar un commit en el repositorio compartido, se ha de obtener el permiso de los demás usuarios habituales del repositorio (los miembros del equipo correspondiente) para minimizar las posibles incongruencias (aunque ya se definen las tareas teniendo esto en cuenta), y no se han de revisar los commits ya que se confía en la capacidad y buen criterio de los miembros del equipo.

Ante la detección de **incidencias**, el que la haya detectado tiene la responsabilidad de encontrar a la persona que haya generado la incidencia, que llamaremos a partir de ahora el incidente. Será responsabilidad del incidente decidir el plan de actuación para solventar la incidencia, en tanto a si puede ser el único responsable y si es más prioritario o no que el trabajo actual que haya que realizar.

Las actividades de **control de calidad** serán variadas. Se realizarán pruebas por separado de frontend y backend y posteriormente se realizarán pruebas de la conjunción de ambos. Para las pruebas del backend se cuenta en primera instancia con tests para comprobar el correcto funcionamiento de las funciones. Para probar el juego completo, se propone:

1. Probar todos los arcos del grafo de juego forzando los escenarios correspondientes y eliminando el azar.
2. Simular partidas con dos agentes jugando entre ellos.

Para las pruebas del frontend,

1. En primera instancia, se comprobará manualmente la correcta inclusión del código en el sistema existente.
2. Se documentará mínimamente las entradas y salidas esperadas para la interacción con el juego (tirada de dados, compra de propiedad, pago de alquiler...) y se comprobará sistemáticamente la corrección del cliente correspondiente.

Para el código a alto nivel, se contrastará con la persona que realice los diagramas y los autores del código correspondiente que el comportamiento del sistema coincide con lo expresado en el diagrama. Tanto la realización de las pruebas como el despliegue del sistema se realizará durante días designados para la conciliación entre backend y frontend, como expresado en el diagrama de Gantt de la Figura 1. Se especificarán las fechas concretas para realizar las actividades al comienzo

de la semana correspondiente a estas actividades. Por tanto, descartamos tanto metodologías de integración como de despliegue continuos.

Se entregará al cliente lo siguiente a la finalización del proyecto:

- Todo el código desarrollado.
- Documentación de despliegue y ficheros de configuración y Docker Compose para el despliegue.
- Un AppImagen con la versión escritorio, ya compilado.

3.3. Calendario del proyecto, división del trabajo, coordinación, comunicaciones, monitorización y seguimiento

La responsable del calendario del proyecto, coordinación, y comunicaciones es Julia, la directora del proyecto, apoyada en los coordinadores backend y frontend para una mejor descripción y ordenación de las tareas, la división del trabajo y la monitorización y seguimiento de las tareas.

Para la **monitorización y medición del progreso**, cada lunes y viernes se revisarán las tareas realizadas por los miembros del equipo correspondiente. Se mira la consecución de la tarea, y de haber quedado mucho por realizar, dado que confiamos en el buen trabajo de los miembros del equipo, concluiremos que el retraso se ha debido a un reparto no equitativo y se volverá a repartir la tarea faltante entre los miembros del equipo, para realizarse en esa semana, antes que las tareas de dicha semana.

Se ha intentado realizar un plan lo suficientemente holgado como para que estas desviaciones, de no prolongarse durante más de una semana, no supongan un problema. De darse continuadamente este retraso, a una semana vista de la entrega se replantearán las tareas a realizar, minimizando estas.

Sobre el **reparto y definición de tareas**, primero se ha realizado una planificación global por requisitos como expresado en el diagrama de Gantt de la Figura 1. Para ello, se ha entendido que las etapas de calidad se producirán cada vez que se suba código a los repositorios y se ha planificado un diseño iterativo y gradual por requisito. A más bajo nivel, estas se realizarán después de la monitorización del progreso, listando el desglose de tareas necesarias para consecuir los objetivos semanales correspondientes al diagrama de Gantt de la Figura 1. Los responsables de las tareas son como lo descrito en la Tabla 1. Del diagrama de Gantt podemos destacar los siguientes hitos internos:

- Entrega intermedia: 6 de abril de 2026. Para esta no se planea haber conseguido completar ningún requisito, pero sí tener listo a falta de integración y despliegue la implementación correspondiente a los requisitos 1, 2, 3, 4, 6 y 9.
- Comienzo de la integración: 3 de abril de 2026.
- Comienzo del despliegue: 15 de abril de 2026.
- Presentación del sistema: 4 o 5 de mayo de 2026.
- Muestra del sistema al cliente: semana del 11 de mayo de 2026.
- Entrega final: 27 de mayo de 2026.

Una vez se hayan desglosado las tareas y concordado cómo se van a realizar (todo aquello que suponga cohesión, como interfaces consistentes o comunicación entre elementos), se repartirán entre los miembros del equipo (backend, web y escritorio), primando la equidad, los conocimientos y habilidades de los miembros, y sus preferencias. Cada uno informará de sus preferencias y al llegar al consenso entre todos se fijarán las tareas con su responsable como Github issues.

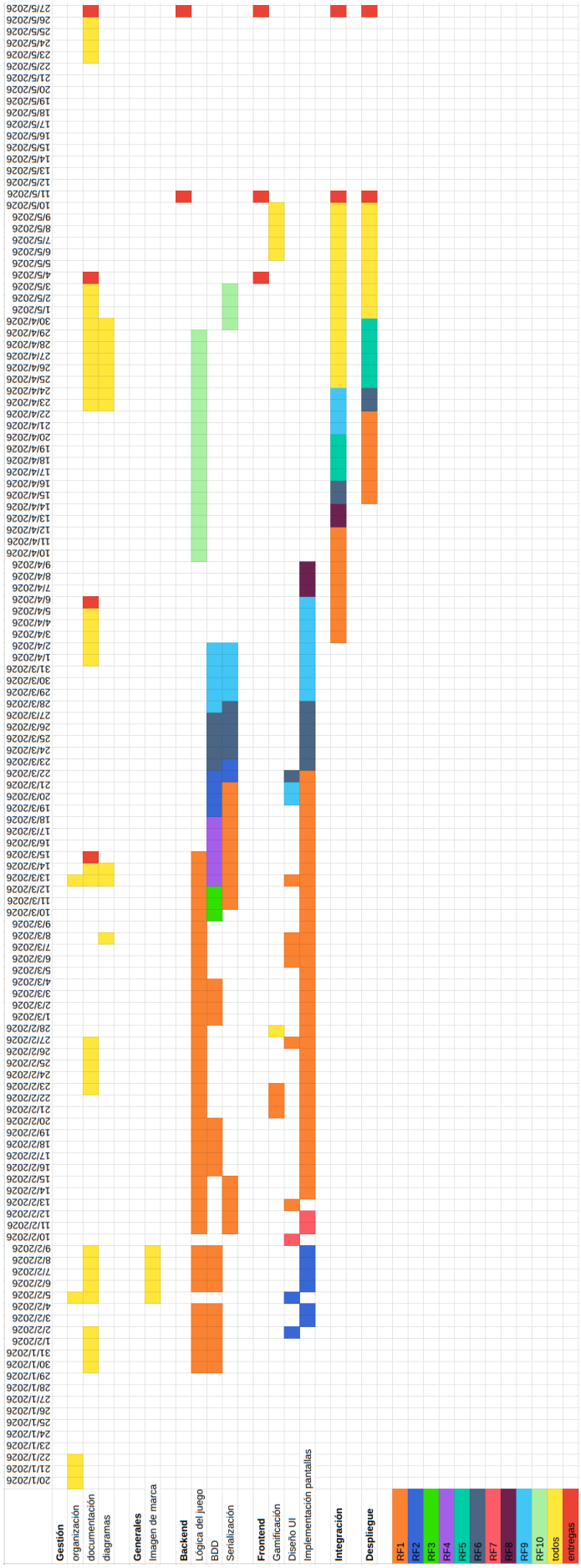

Figura 1: Diagrama de Gantt con la planificación del proyecto.

Para la **comunicación interna**, se han creado grupos de WhatsApp por módulo y por equipios, así como un canal de Discord análogo para comunicaciones y cuestiones de rápida resolución o inmediatas, y se realizarán reuniones de seguimiento entre todos los equipos durante las sesiones de prácticas, así como las reuniones dentro del equipo cuando los miembros lo estimen oportuno. Las reuniones de las sesiones de prácticas servirán para actualizar los planes en caso de desviarse del inicial. Las decisiones tomadas acabarán reflejadas en Github issues o en la documentación (o en la carpeta compartida de Drive a modo de borrador, de ser algo extenso, de forma provisional hasta aparecer en la documentación), por lo que no se utilizan documentos adicionales con actas.

Ante diferencias en la **gestión del equipo humano**, se plantea la inclusión de mediadores imparciales o no involucrados en la disputa inicial para intentar consensuar a las partes. En caso de no resolverse el conflicto, se pospondrá la discusión a cuando se halla enfriado, no siendo más de 2 semanas.

4. Análisis y diseño del sistema

En esta sección se detallan los requisitos y diseño del sistema.

4.1. Análisis de requisitos

En la Tabla 2 se recoge la relación de requisitos funcionales del sistema, y en la Tabla 3 se recogen los requisitos no funcionales del sistema.

4.2. Diseño del sistema

El sistema ofrece un cliente web y un cliente escritorio, para los que utiliza una arquitectura en 4 y 3 niveles, como puede verse en el diagrama simplificado de la Figura 2.

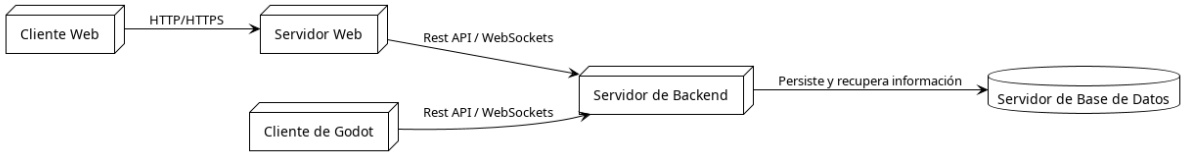

Figura 2: Diagramas de la arquitectura a alto nivel.

El sistema cuenta con un servidor de bases de datos PostgreSQL, un servidor con el backend para la lógica del juego y demás peticiones, y para el caso del cliente escritorio, basta con un único cliente, mientras que para la web necesita un servidor web y sobre este el cliente web con nuestra aplicación. El diagrama de componentes y conectores de la Figura 6 (ver Apéndice A.1) nos concreta el diagrama de la Figura 2 y las comunicaciones entre los elementos desplegados. La comunicación se realiza a través de web sockets para el juego y de API Rest para el resto de la aplicación.

Además aportamos, para explicar la lógica y secuenciación del juego, el diagrama de estados de la Figura 3, que representa un turno de juego y sus fases. El turno comienza con la fase de tirar los dados y después de resolverse lo ocurrido en la casilla a la que se cayó se pasa a la fase de negocios, en la que pueden realizarse opcionalmente y en cualquier orden las transacciones del juego (intercambio, y administrar propiedades: construcción, derruir casas e hipotecar), y por último hay una fase de liquidación análoga a la anterior para que el jugador evite quedarse en números rojos. Los diagramas de secuencia de estas dos últimas fases se encuentran en la Figura 4. Estos muestran la transacción más general (la hipóteca), en la que proponente y receptor pueden ser distintos.

Otro aspecto del turno por describir es la subasta, en la que se esperará a que todos los jugadores seleccionen el valor por el que subastar (o no participar) para continuar. El diagrama de estados es muy sencillo, se recoge en la Figura 7 (ver Apéndice A.2).

El proyecto está organizado en 3 grandes repositorios de código: el backend, el cliente web y el cliente escritorio. La estructura de estos repositorios, en cuanto a diagramas y vistas de módulos, siguen la estructura habitual de proyectos con estas tecnologías, y pueden consultarse en los diagramas de las Figuras 8, 10 y 9, respectivamente (ver Apéndice A.3).

El despliegue del sistema se realizará en una misma máquina, utilizando varios contenedores, con un sistema en dos capas, sin perjuicio a que se pudiera realizar en varias máquinas distintas cada contenedor, en cuatro capas, según lo especificado en los diagramas de despliegue de la Figura 5.

ID	Descripción	Prioridad
RF-1.1	El sistema permitirá jugar a Magnate, siguiendo las reglas descritas en el Apéndice C.	M
RF-1.2	El sistema mostrará los resultados de una partida una vez finalizada, incluyendo el capital final de los jugadores y los beneficiarios de los bonus finales aplicados.	M
RF-2.1	El sistema debe permitir a los usuarios registrarse con un nombre de usuario único y una contraseña.	M
RF-2.2	El sistema exige para el registro de una cuenta de usuario nueva que se confirme la contraseña.	M
RF-2.3	El sistema debe permitir a los usuarios iniciar sesión introduciendo su nombre de usuario y su contraseña.	M
RF-3	El sistema guardará la información de las partidas en curso necesaria para reanudarlas.	M
RF-4.1	El sistema almacenará el historial de partidas de los usuarios.	M
RF-4.2	El sistema permitirá a los usuarios consultar su historial de partidas completas.	M
RF-4.3	El historial de partidas recogerá para cada partida la posición en la que quedó el usuario tras jugarla y las monedas que ganó.	M
RF-5	Las partidas se pueden pausar y reanudar en otra plataforma.	M
RF-6.1	El sistema permitirá a los usuarios que hayan iniciado sesión observar su propia información ligada a un usuario.	M
RF-6.2	La información ligada a un usuario incluye: nombre de usuario, skin de ficha predeterminada para jugar y estadísticas de sus partidas.	M
RF-6.3	Las estadísticas de las partidas ligadas a un usuario incluyen: nº partidas jugadas, nº partidas vencidas, monedas para la tienda.	M
RF-6.4	El sistema permitirá al usuario elegir su skin de ficha predeterminada entre las que posea.	M
RF-7.1	El sistema permite jugar partidas en dos modalidades: partida pública y partida privada.	S
RF-7.2	Las partidas privadas permitirán que jueguen los poseedores de un código de la sala privada y/o con bots, sumando hasta 4 jugadores.	S
RF-7.3	Las partidas públicas realizan un emparejamiento entre los usuarios por orden de llegada, agrupando 4 jugadores siempre que haya suficientes en línea.	S
RF-7.4	La moneda del juego se recibe tras jugar cada partida como el capital remanente al terminar la partida y aplicar los bonus.	S
RF-8.1	El sistema permitirá a los usuarios intercambiar mensajes en lenguaje natural durante el transcurso de una partida.	C
RF-8.2	El sistema permitirá a los usuarios incluir emoticonos en los mensajes del chat durante el transcurso de una partida.	C
RF-9.1	Se pueden adquirir distintos skins para las fichas del juego en la tienda.	S
RF-9.2	El sistema permitirá a los usuarios adquirir distintos skins para las fichas de las partidas de Magnate en la tienda, intercambiando la moneda del juego, que se obtiene jugando partidas.	S
RF-9.3	Se pueden adquirir distintos emoticonos para emplear en el chat del juego en la tienda.	C
RF-9.4	El sistema permitirá a los usuarios adquirir distintos emoticonos para el chat de las partidas de Magnate en la tienda, intercambiando la moneda del juego, que se obtiene jugando partidas.	C
RF-10	Se diseñará una IA o bots contra la que jugar en las partidas privadas.	C

Tabla 2: Requisitos funcionales del sistema.

RNF-1	El cliente de escritorio debe funcionar en un computador con sistema operativo Arch Linux 6.18.6.
RNF-2	El cliente web debe funcionar en un navegador con versión Chromium 144.0.7559.96.
RNF-3	El sistema es seguro a usos malintencionados de las plataformas.

Tabla 3: Requisitos no funcionales del sistema.

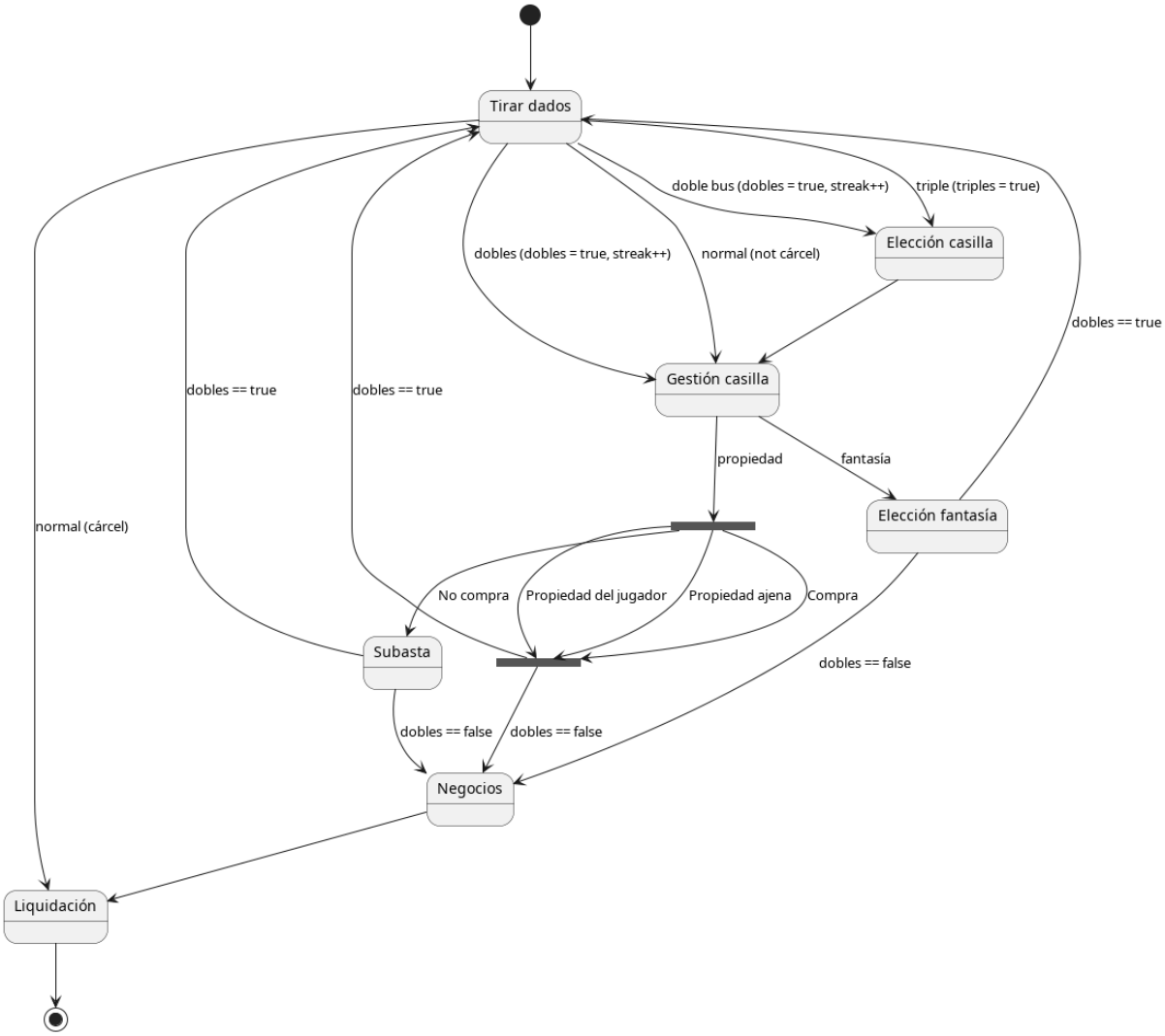

Figura 3: Diagrama de estados de un turno de juego con sus fases.

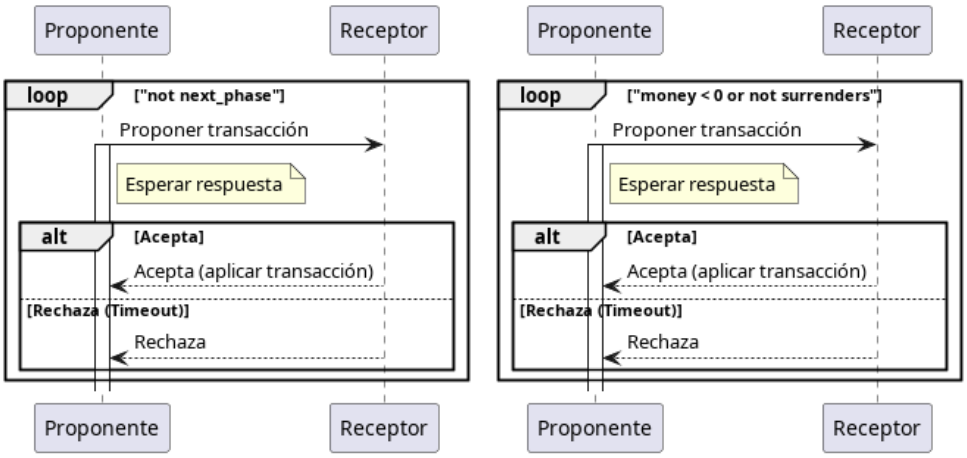

Figura 4: Diagramas de secuencia de las fases de negocios (izquierda) y liquidación (derecha).

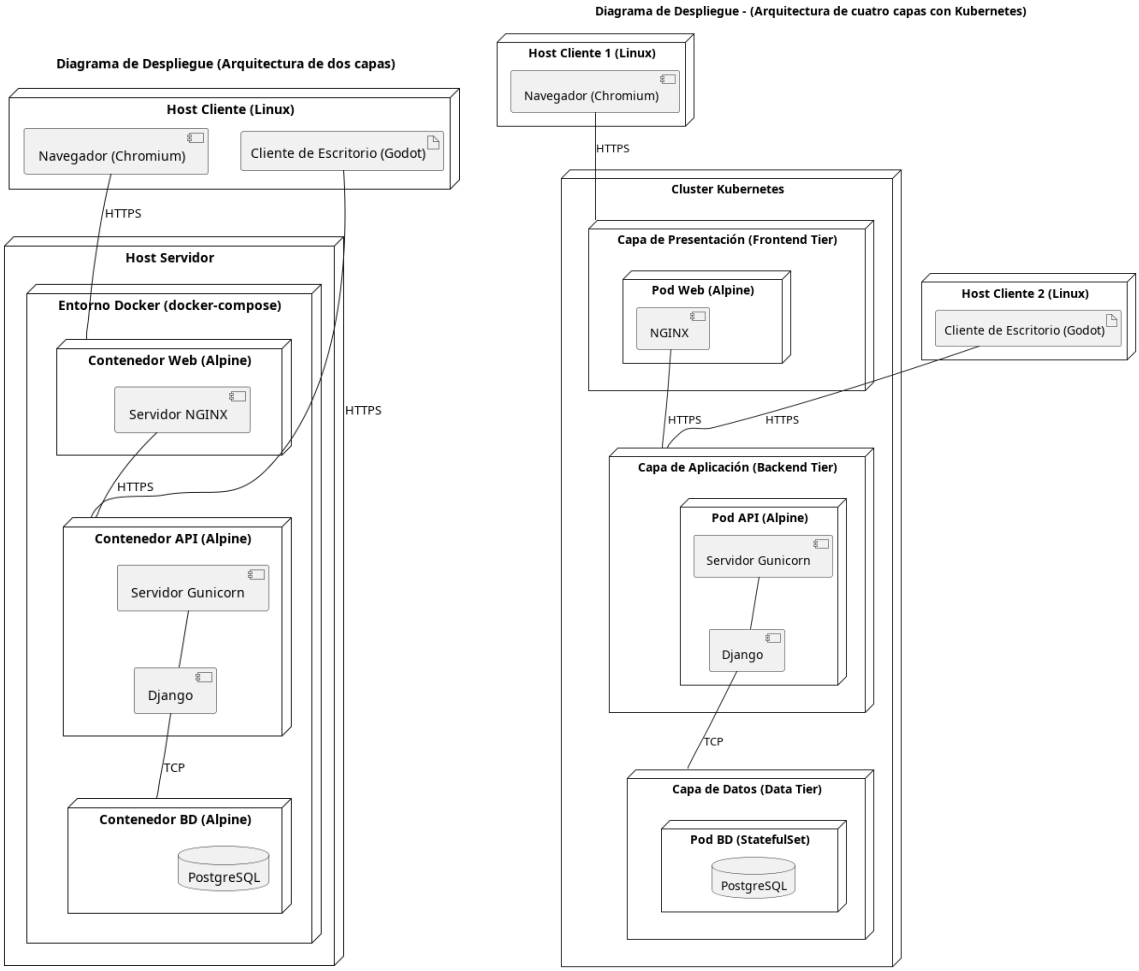

Figura 5: Diagramas de despliegue. A la izquierda, diagrama en 2 capas, como se realizará el despliegue, y a la derecha, en 4 capas, señalando las posibilidades de despliegue.

4.2.1. Decisiones de diseño

Las primeras decisiones de diseño se tomaron sobre las **tecnologías** a usar en el proyecto. Las posibles candidatas consideradas son todas tecnologías libres y sin licencia comercial. Para las tecnologías web se barajaron html con css y React. Aunque los miembros del equipo tenían experiencia con html y css, la dificultad de realizar un proyecto tan complejo en estos lenguajes nos llevó a decidirnos por React, cuya curva de aprendizaje es bastante menor una vez conocidas estas tecnologías, y cuenta con una forma fácil de incluir paquetes de componentes. Además, React cuenta con numerosas librerías sobre las que construir un juego. Se desestimó la librería para juegos boardgame.io ya que esta supliría el trabajo del que se encarga el backend, pero sí se utilizó Phaser para gráficos, componentes y elementos del juego, ya que el aprendizaje merecería la pena frente a realizar todo en React. Por motivos similares, se desestimó utilizar directamente HTML Audio para gestionar el sonido y se utilizó la librería Howler, que parecía más sencilla que otras del estilo. Como herramienta para la documentación, se barajaron react-docgen, TypeDoc y Storybook, y se decidió usar TypeDoc por ofrecer documentación automática para todos los archivos (clases, interfaces, funciones y módulos además de los componentes).

Como segundo cliente se decidió el escritorio sobre el móvil porque facilitaría las labores de interfaz, al no tener que conciliar diseños horizontales y verticales. Como nueva tecnología para el escritorio se optó por Godot, un motor orientado al desarrollo de juegos que tiene una interfaz para el desarrollo intuitiva y cuenta con portabilidad a una amplia gama de dispositivos, por lo que se presentaba como una buena herramienta para aprender. Otras tecnologías consideradas fueron Draft y Flutter, que se descartaron por no estar dirigidas a juegos. Para la herramienta para generación de documentación del proyecto en Godot se eligió Godoct por ser compatible con Godot 4, no como Doxygen, que tiene las mismas características, y por ser más automática que las demás consideradas (MkDocs y Sphinx) y bastar con seguir el estilo y forma de poner comentarios para obtener la documentación.

Como tecnologías de backend se barajaron Django, Flask, NextJS y FastAPI, y se decidió usar Django por tener experiencia previa en la tecnología. Además, salvo Flask, excesivamente sencilla, las tecnologías mencionadas ofrecen funcionalidades similares. Para la representación de la información se consideraron los formatos yaml, xml y JSON, y finalmente se decidió utilizar ficheros JSON por su uso extendido y generalizado, y la experiencia previa trabajando con ellos. Para la base de datos, se barajaron bases de datos SQL, que son con las que tenemos experiencia, y nos decantamos por PostgreSQL por ser compleja, potente y eficiente. La decisión de utilizar las herramientas automáticas de generación de documentación vienen por haberse encontrado tras una pequeña búsqueda, ver que estas bastarían para el proyecto, estar familiarizados con el formato docstrings, poder integrarse y generar pdfs de forma sencilla y que mkdocs es moderno y fácil de configurar.

Otra decisión de diseño tomada fue utilizar el servidor de Mario Casado para el despliegue. Se optó por esta opción sobre cualquier otra plataforma online para el despliegue debido a su acceso gratuito y sin condiciones, y a la familiaridad de algunos de los miembros del equipo con dicho entorno.

La decisión sobre el diseño de la **arquitectura** viene directamente motivadas por las tecnologías empleadas y los flujos de trabajo habituales necesarios para desplegar aplicaciones que utilizan dichas tecnologías. Como servidor web sobre el que desplegar nuestro cliente web se decidió utilizar nginx en lugar de Apache, ya que contábamos con experienca previa en dicho servidor y sus prestaciones son suficientes para el proyecto.

Sobre los **protocolos de comunicación** entre los componentes del sistema, se optó por una API Rest para las comunicaciones no realcionadas con el juego, ya que son más sencillas, y web sockets para poder almacenar el estado del juego y tratar las comunicaciones de frontend a backend (con los objetos responses) y de backend a frontend (con actions, enviando el contenido en un JSON serializado para facilitar el tratamiento en el frontend).

5. Memoria del proyecto

En este capítulo se describe el ajuste a la realidad de lo previsto en el plan de la Sección 3.

5.1. Inicio del proyecto

La elección inicial de tecnologías permitió que el proyecto avanzara a buen ritmo una vez nos familiarizamos con ellas, siguiendo el plan inicial.

5.2. Ejecución y control del proyecto

A día 15 de marzo, la comunicación interna se ha concentrado más en los grupos de mensajerías para los equipos reducidos (backend, cliente web y cliente escritorio) y en persona para la medición del progreso y la comunicación de la empresa al completo. Creemos que esto refleja las necesidades de las actividades que hemos estado realizando hasta el momento y el primer periodo de adaptación al trabajo en un grupo tan grande, y que conforme avance el proyecto usaremos el resto de herramientas que hemos preparado. Lo mismo sucede con las pruebas: de momento solo se han realizado tests automáticos para el backend, que han resultado muy útiles, y pruebas manuales en el frontend.

En cuanto al reparto de tareas, la duplicación del trabajo en frontend y ser la primera vez que trabajábamos juntos algunos de los integrantes hizo que nuestro planteamiento inicial del reparto de trabajo derivara en tener que repetir el trabajo por diferencias en diseños. Esto provocó que tuviéramos más trabajo durante un par de semanas, y nos retrasásemos dos semanas con respecto al plan inicial, que era demasiado optimista. Tomamos medidas el 27 de febrero cambiando progresivamente nuestra manera de repartir el trabajo en el frontend: en vez de asignar unas pantallas a web y otras a escritorio, decidimos asignarlas en primer lugar a web, ya que la plataforma es más restrictiva que escritorio, por si era necesario hacer algún pequeño cambio con respecto al diseño inicial consensuado, y después escritorio copiaría las pantallas. El cambio debido a esta divergencia se dio por completo el 13 de marzo, cuando revisamos el plan inicial, que tenía dos fases de integración y despliegue por requisitos, y lo cambiamos al plan actual, que se centra primero en el diseño e implementación y después en la integración y despliegue. El plan también favorece el reparto de los requisitos por persona, permitiendo el trabajo paralelo en mayor medida. Con estas medidas esperamos que no sucedan más retrasos.

Las herramientas y tecnologías empleadas, así como los procesos para conocer el trabajo realizado y pendiente y sus encargados, y los procesos para la implementación y procesos relacionados se han seguido respecto al plan, con resultados satisfactorios. Destacar que todavía no se ha tenido oportunidad de probar los procesos de integración ni despliegue.

5.3. Cierre del proyecto

A día 15 de marzo, el proyecto sigue el plan descrito en la Figura 1, y los esfuerzos realizados están en torno al 33 % del proyecto, adecuado para el momento en el que nos encontramos.

Durante este tiempo se han implementado las estrategias de mitigación de riesgos como planteado. Los riesgos 3, 7, 9, 10 y 11 todavía no se han presentado, y los riesgos 1, 5, 8 y 4 se han presentado sin efectos negativos (aunque todavía están a tiempo de mostrarlos, sobre todo el último). El riesgo 6 se presentó muy temprano en el proyecto, en torno al 23 de febrero, y fue solventado sin pasar más de una semana. Con respecto al riesgo 2, su aparición y efectos se explicaron en la subsección 5.2.

Además de los riesgos, surgió un pequeño imprevisto entre los miembros del equipo backend: al juntar código de distintos miembros en el entorno de desarrollo de Mario se veía algo de ruido en los ficheros (por cambio de formato y demás). Así que todos los miembros del equipo pasaron a usar el mismo entorno de desarollo (VisualStudio, de los que ya estaban contemplados). No supuso un retraso de más de un día. Este imprevisto podría considerarse como riesgo en futuros

proyectos, imponiendo como mitigación que las personas que editen los mismos ficheros lo hagan en el mismo entorno de desarollo.

Hemos aprendido las siguientes lecciones:

- Usar el mismo entorno de desarrollo para editar los mismos ficheros evita problemas menores pero molestos.
- Al trabajar con distintas herramientas de forma que sea necesario replicar el trabajo puede ser una buena práctica ir más adelantado en una plataforma que en otra para evitar repetir trabajo, si una de estas tecnologías es más restrictiva que la otra.
- La comunicación en el equipo depende de las actividades que realicen, afectando tanto al contenido como a su forma.

5.3.1. Dedicación por integrante

En la Tabla 4 recogemos las horas empleadas por cada miembro de la empresa en cada tarea. Podemos comprobar que estas se corresponden con las tareas que tenían asignadas en un principio, están equilibradas teniendo en cuenta las actividades realizadas hasta la fecha y suponen en torno a un % del proyecto.

Hugo Naudín	
Coordinación backend	
Lógica del juego	
Bases de datos	
Serialización	
Documentación y diagramas	
Despliegue	
Jaime Alconchel	35
Coordinación backend	10
Lógica del juego	
Bases de datos	
Serialización	
Documentación y diagramas	
Despliegue	
Mario Casado	
Coordinación backend	
Lógica del juego	
Bases de datos	
Serialización	
Documentación y diagramas	
Despliegue	

Tabla 4: Dedicación: horas y tareas realizadas por los integrantes de la empresa.

Referencias

[1] Godot Engine contributors. *Best practices*. Godot Engine official documentation. Godot Engine Documentation. 2024. URL: https://docs.godotengine.org/en/stable/tutorials/best_practices/index.html (visitado 14-03-2026).

[2] Hasbro. *Monopoly: Reglas de juego (Edición en español)*. Accedido el 2 de febrero de 2026. URL: https://www.hasbro.com/common/instruct/Monopoly%28Spanish%29.pdf.

[3] Newwby. *Godoct*. Repositorio de software. GitHub, 2024. URL: https://github.com/newwby/Godoct (visitado 14-03-2026).

[4] Guido van Rossum, Barry Warsaw y Alyssa Coghlan. *PEP 8 – Style Guide for Python Code*. Python Enhancement Proposal. Python Software Foundation. 5 de jul. de 2001. URL: https://peps.python.org/pep-0008/ (visitado 14-03-2026).

A. Diagramas

A.1. Diagrama de componentes y conectores.

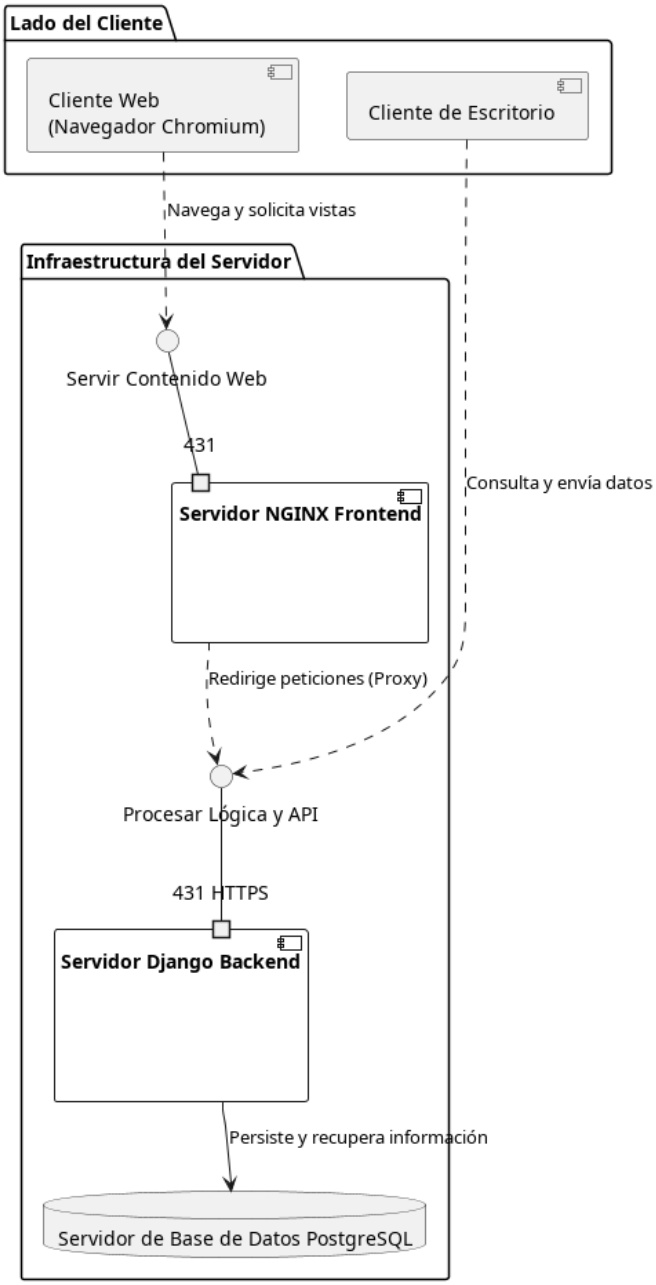

Figura 6: Diagrama de componentes y conectores.

A.2. Diagrama de estados de la fase de Subasta de un turno del juego Magnate

Figura 7: Diagrama de estados de la fase de Subasta de un turno del juego Magnates.

A.3. Diagramas de módulos

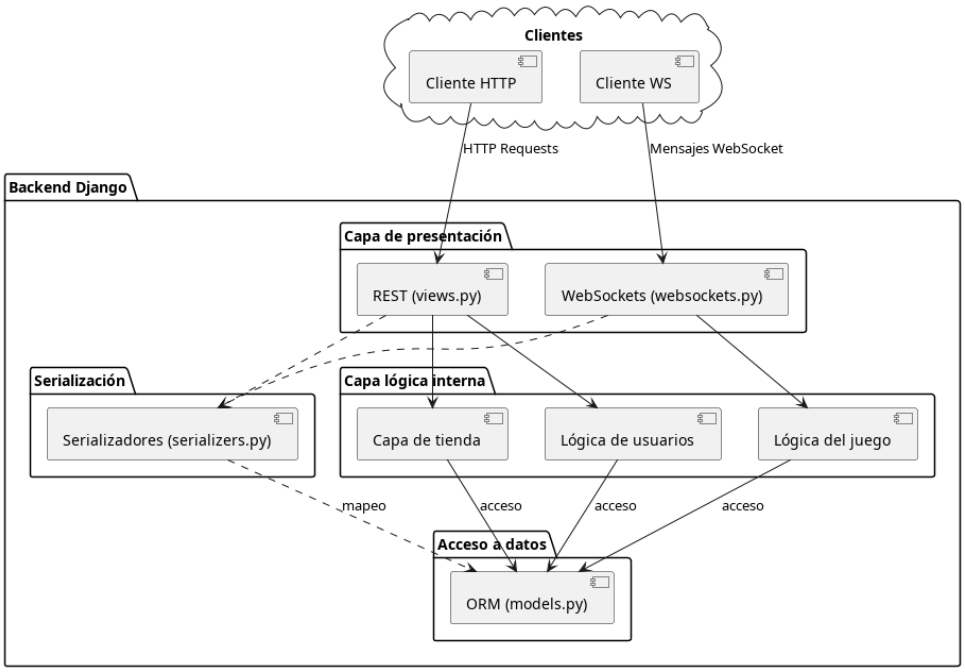

Figura 8: Diagrama de módulos del Backend.

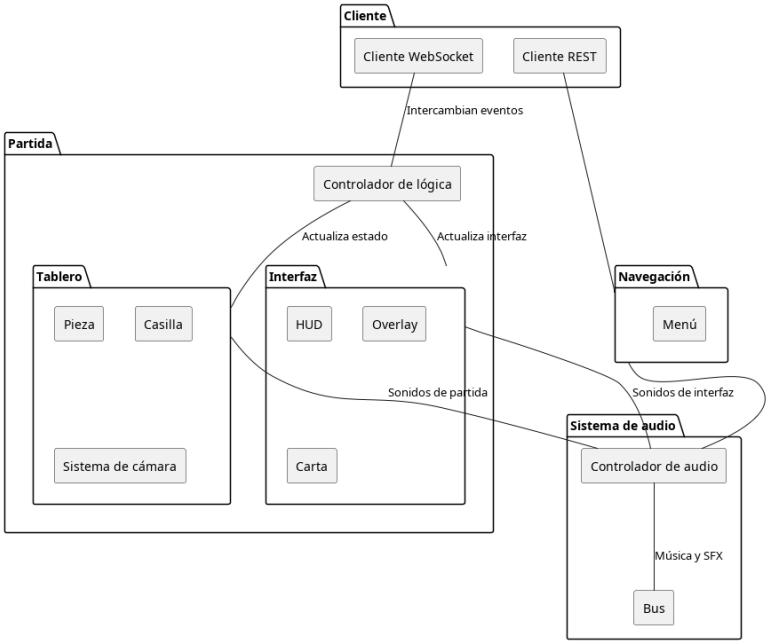

Figura 9: Diagrama de módulos del cliente de escritorio.

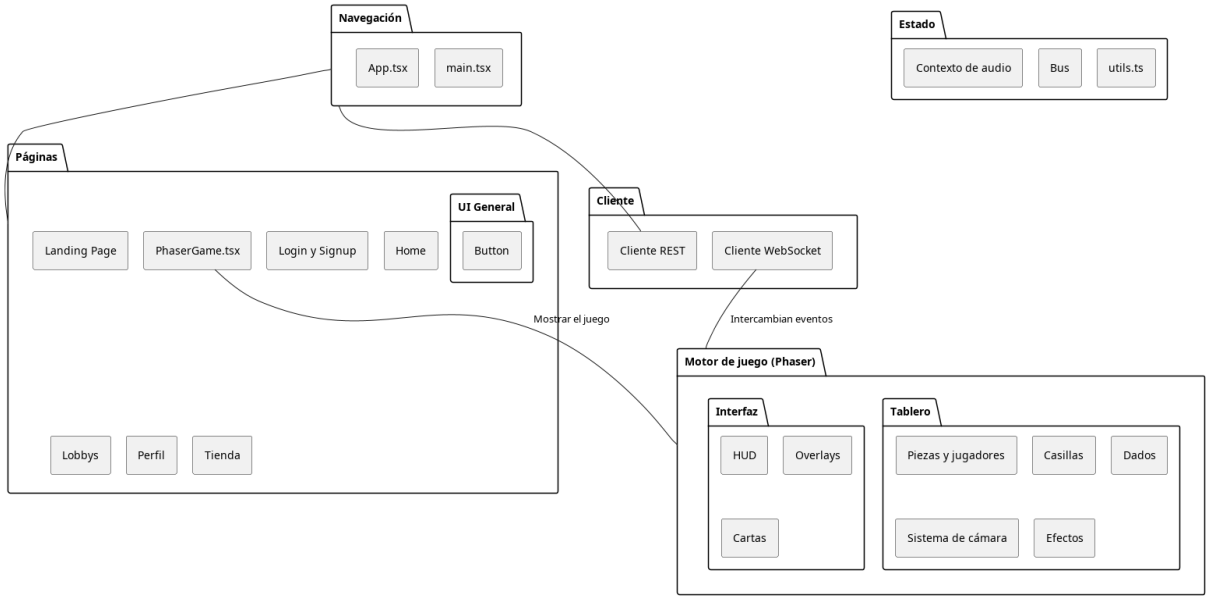

Figura 10: Diagrama de módulos del cliente web.

B. Análisis de riesgos

En la Figura 11 se recoge el análisis de riesgos detallado realizado por la empresa.

Proyecto Software
Análisis de Riesgos del Proyecto
Ejercicio de clase: Identificación y análisis de riesgos del proyecto

Código	Descripción	Riesgo S/N	Probab. 1--5	Impacto 1--5	Exp. al riesgo Prob X Imp
Ri1	Hay integrantes del equipo que tienen muchos proyectos (y/o asignaturas) a la vez	S	4	3	12
Ri2	La falta de experiencia trabajando juntos puede afectar al éxito del proyecto	S	1	3	3
Ri3	La toma de decisiones se puede bloquear por no estar dispuestos los miembros del equipo a tomarlas (asumir esa responsabilidad)	S	2	2	4
Ri4	Nunca se ha trabajado en un grupo tan grande	S	4	2	8
Ri5	El conocimeinto o desconociemiento del cliente (profesor)	S	3	2	6
Ri6	El proyecto cuenta con unas funcionalidades que no están muy claramente definidas	S	3	3	9
Ri7	Nunca se ha construido un sistema tan complejo como el propuesto	S	4	3	12
Ri8	Se ha elegido una tecnología que se conoce poco o nada	S	4	2	8
Ri9	Se ha trabajado, o no, con el entorno de despliegue elegido	S	2	2	4
Ri10	La fecha de entrega está muy lejos en el tiempo	S	3	2	6
Ri11	El proyecto hay que desarrollarlo en dos fases	S	3	2	6
Ri12	El proyecto requiere la entrega de un sistema funcionando y una documentación complementaria	N			

Total de riesgos		**11**
	Altos	2
	Medios	6
	Bajos	3

RE > 9
RE entre 4 y 9
RE <= 4

Cuantificar probabilidad de que se presente el problema
1: < 10%
2: alrededor de un 30%
3: alrededor de un 50%
4: alrededor de un 90%
5: > 90%

Cuantificar impacto si se presenta el problema
1: dos días de retraso
2: una semana de retraso
3: unas dos semanas de retraso
4: unas tres semanas de retraso
5: cuatro o más semanas de retraso

Figura 11: Análisis de riesgos.

C. Reglas

C.1. Introducción

En este documento se definirán las mecánicas y reglas del juego *Magnate*. Se explicarán las diferentes fases del juego, las acciones que pueden realizar los jugadores y las condiciones de victoria.

Se definirá desde el diseño del tablero hasta los distintos eventos especiales que pueden ocurrir, considerando todos los casos posibles.

Las reglas están basadas en las reglas originales del Monopoly [2] tomando inspiración adicional de la saga de videojuegos Mario Party. Magnate busca ser un juego dinámico con alto grado de aleatoriedad, pero sin perder la estrategia del Monopoly original.

C.2. Objetivo del juego

El objetivo del juego es ser el jugador con mayor valor neto al final de la partida, limitada por un número de rondas. El valor neto de un jugador se calcula mediante la suma de su dinero en efectivo más el valor de venta de todas sus propiedades, sin contar las que tenga hipotecadas y el valor de venta de sus construcciones.

C.3. Equipamiento

A continuación está la lista de elementos para una partida.

- Tablero
- 2 Dados normales
- 1 Dado bus
- Cartas de las propiedades
- Casas
- Hoteles
- Cartas "Sales de la cárcel"
- Cartas de evento fantasía

C.3.1. Los dados: definición y funcionamiento

En el juego hay 3 dados en juego, 2 normales de 6 caras numerados del 1 al 6 y uno especial llamado *Dado bus*. El Dado bus está compuesto de 6 caras, 3 de ellas con el dibujo de un bus y las otras 3 numeradas del 1 al 3.

En una tirada de dados se tiran siempre los 3 dados a la vez. En caso de salir 3 números se tendrá en cuenta para el movimiento la suma total de los 3. En caso de salir un bus el jugador podrá elegir 3 valores: el valor de uno de los dados normales, el valor del otro de los dados normales o la suma de los 2 dados normales.

Para las mecánicas de los dobles se tendrán en cuenta únicamente los dados normales. Al sacar dobles, el jugador gana una tirada extra. Si llega a sacar dobles una tercera vez consecutiva en el mismo turno, va directamente a la cárcel.

Si en la tirada sale un triple (en los casos en los que el Dado bus saque número), el jugador puede viajar a la casilla del tablero que quiera, aplicándose el efecto de caer en esa casilla tal y como hubiera sido si la tirada hubiera sido convencional. Después de sacar un triple no se vuelve a tirar (como si de unos dobles se tratara). Los triples no cuentan en la racha de dobles, por lo que no te pueden mandar a la cárcel.

C.3.2. Tablero

El tablero está compuesto de 2 pistas cuadradas concéntricas: el anillo exterior tiene 36 casillas y 4 esquinas y el anillo interior tiene 12 casillas y 4 esquinas. La particularidad de las casillas para transición y cómo afectan al tablero se comentará con las casillas de puentes.

C.4. Casillas

En esta sección se definirán todos los distintos tipos de casilla que hay en el tablero y sus efectos y mecánicas.

C.4.1. Propiedades

Las propiedades son casillas que se pueden comprar, vender e hipotecar, y por las que se cobra alquiler a jugadores rivales. Si un jugador cae en una de estas casillas pueden pasar varias cosas en función del estado de compra de la propiedad:

- La casilla no tiene dueño. En este caso el jugador tiene derecho a comprarla por el precio de compra indicado. En caso de no querer comprar o no tener suficiente dinero, la propiedad pasa a ser subastada.
- La casilla tiene dueño. El jugador debe pagar el alquiler calculado en función de las construcciones o de si tiene el grupo completo.
- La casilla está hipotecada. No ocurre nada.
- La casilla es del propio jugador. No ocurre nada.

C.4.2. Grupos de propiedades

Las propiedades están divididas en grupos de color. Un jugador que posea todas las propiedades de un mismo grupo, podrá cobrar el doble de alquiler por las propiedades sin construcciones y obtendrá derecho a construir en ellas.

C.4.3. Puentes

Los puentes son un tipo especial de propiedad. En el juego tienen una función doble: poder ganar dinero por alquiler (como el resto de propiedades), y habilitar la transición entre el cuadrado interior y el cuadrado exterior.

El alquiler a pagar en los puentes viene marcado por el número de puentes que el jugador posea.

La mecánica de transición viene marcada por el valor usado en la tirada de dados (las casillas que el jugador elige mover). Si la tirada es par se continúa en el mismo cuadrado, si es impar se cambia de cuadrado (o viceversa, lo que indique la casilla).

En caso de caer justo en la casilla del puente, la decisión par o impar la tomará la tirada de dados de la siguiente jugada y no la tirada de la jugada de caer en el puente.

Las casillas puente, a pesar de ocupar el espacio de 2 casillas convencionales únicamente contarán como una en los movimientos tanto en los casos de mantenerse en el cuadrado como en el caso de transicionar al otro cuadrado.

C.4.4. Servidores

Los servidores son otro tipo especial de propiedad. El valor de su alquiler viene determinado por la cantidad de servidores que el jugador posea.

C.4.5. Casillas fantasía

Al caer en una casilla fantasía, aparecerán 2 cartas, una desvelada y una oculta. La carta desvelada tendrá un evento fantasía aleatorio, que se podrá aplicar por el precio indicado en la carta. El evento fantasía de la carta oculta es gratuito. En caso de no querer comprar el evento desvelado, o no tener dinero para comprarlo, habrá que escoger la carta oculta obligatoriamente. En caso de comprar y aplicar el evento fantasía de pago, no se aplicará ni desvelará el evento oculto.

Los eventos fantasía tendrán acciones positivas (como ganar dinero), negativas (como perder propiedades) o neutras (como mover a otro jugador a cualquier casilla del mapa).

C.4.6. Vas a la cárcel

Si caes en esta casilla vas directamente a la cárcel y acaba tu turno. En el caso de que el jugador deba dinero al banco se le permitirá realizar las gestiones necesarias para saldar su deuda.

C.4.7. Cárcel

Un jugador en la cárcel tiene derecho al principio de su turno a tirar los dados. Si saca dobles puede salir de la cárcel usando el valor de la tirada y jugar su turno con normalidad (pero sin repetir turno por ser dobles). Si no saca dobles, puede pagar una fianza de valor fijo. Si al tercer turno en la cárcel no ha salido, será obligado a pagar la fianza.

Los jugadores en la cárcel tienen derecho a cobrar alquileres y a realizar acciones de gestión en su turno, pero no pueden moverse por el tablero ni participar en las subastas.

Los turnos en los que un jugador acabe en la cárcel acaban cuando se entre en la cárcel sin tener oportunidad de hacer gestiones (excepto el caso en el que tenga que saldar deudas con el banco).

C.4.8. Salida

En esta casilla comienza la partida y cada vez que un jugador pasa por ella (al caer justo sobre ella o la primera tirada en la que la dejas atrás de la vuelta al cuadrado correspondiente) recibe una cantidad fija de dinero.

Si por algún motivo un jugador decide mantenerse en el cuadrado interior y no pasar por salida, no recibirá dinero.

C.4.9. Tranvías

Hay 4 estaciones de tranvía repartidas por las esquinas del tablero. Si un jugador cae en una de estas casillas podrá moverse directamente a otra estación de tranvía de su elección pagando un precio por el desplazamiento.

C.4.10. Párking

Durante la partida, las multas, pagos de impuestos y demás ingresos a la banca en general, se acumularán en un bote. El jugador que caiga en el parking se llevará el bote completo. Si el bote está vacío, no ocurre nada.

C.5. Preparación

Cada jugador comienza con una cantidad de dinero determinada. El turno se elige tirando un único dado, el que obtenga mayor valor empieza. En caso de empate se vuelven a lanzar los dados para desempatar. El orden de valores indica el orden de juego de mayor a menor.

C.6. Turno de juego

Al comienzo del turno, el jugador lanzará los dados y moverá en función de lo que salga. Al caer en una casilla, se aplicará su efecto correspondiente. Tras todos los efectos de casilla, el jugador podrá realizar acciones de gestión. Estas acciones son: comprar o vender construcciones, hipotecar o deshipotecar propiedades y comprar o vender propiedades a otros jugadores (intercambios, ver la Sección C.10).

Haga lo que haga el jugador, su balance debe quedar positivo al final de su turno, de lo contrario será declarado en bancarrota y eliminado de la partida.

C.7. Subastas

Las subastas serán pujas abiertas de valor oculto. Es decir, cada jugador participante en la puja decidirá si participar o no, y en caso de participar elegirá el valor por el que puja. El resto de jugadores no sabrá en ningún momento los jugadores que deciden participar en la puja ni los valores por los que pujan hasta el final de la subasta, donde se desvelarán los valores y el ganador de la puja. En caso de empate la propiedad no se dará a nadie.

Los jugadores que provocan una subasta no podrán participar en ella.

C.8. Alquileres

Si un jugador cae en una propiedad de otro jugador, deberá pagar el alquiler correspondiente. El alquiler se calcula en función de si el dueño tiene el grupo de propiedades completo y de las construcciones que haya en la propiedad.

En caso de que el jugador no tenga dinero para pagar el alquiler al dueño, el banco pagará el alquiler y al final del turno, el jugador deberá saldar su deuda con el banco mediante el hipotecado de propiedades o la venta de construcciones o propiedades tanto al banco como a otros jugadores.

C.9. Construcciones

Una vez un jugador tiene el grupo completo de un color, podrá construir casas en las propiedades de este grupo. El coste de construcción de cada casa vendrá marcado en cada propiedad. Las casas deben construirse de forma escalonada, es decir, si se quiere construir una segunda casa en una propiedad primero se tendrá que tener una casa en el resto de propiedades del grupo.

Una vez un jugador tenga 4 casas en todas las propiedades de un grupo, podrá sustituir las 4 casas de una propiedad por un hotel por el precio de compra de una casa.

A la hora de demoler construcciones se hará igualmente de forma escalonada en sentido inverso. En el caso particular de los hoteles, no se demolerán de golpe, sino que primero se pasará de hotel a 4 casas. El jugador recibirá la mitad del precio de compra de las construcciones demolidas.

C.10. Intercambios

Cada turno, el jugador podrá proponer intercambios a otros jugadores. Un intercambio se hará entre el jugador que lo propone y otro que podrá elegir. Se podrán intercambiar dinero y cartas (propiedades, servidores y puentes) por dinero y cartas.

No se podrán intercambiar propiedades de un grupo de propiedades en el que alguna de las propiedades tengan construcciones.

C.11. Bonificaciones finales

Al final de la partida, antes de desvelar al ganador, se aplicarán 3 bonificaciones finales aleatorias. Las bonificaciones darán una cantidad de dinero determinada por un porcentaje del valor total de todos los jugadores sumado. Estas bonificaciones vienen determinadas por estadísticas

de la partida, como el jugador que más se haya desplazado o el que más alquileres ha tenido que pagar al resto de jugadores, entre otras.

C.12. Bancarrota

Si un jugador no logra acabar su turno con balance positivo, incluso después de hipotecar sus propiedades, de vender todas sus construcciones y en general después de hacer gestiones para recaudar dinero, todas las propiedades que le queden serán embargadas por el banco, poniéndolas inmediatamente a la venta por su valor original.

Esto se aplicará también a los jugadores que se rindan, cuyo dinero en metálico quedará además absorbido por el banco sin ir al párking gratuito.